

COMP 3311

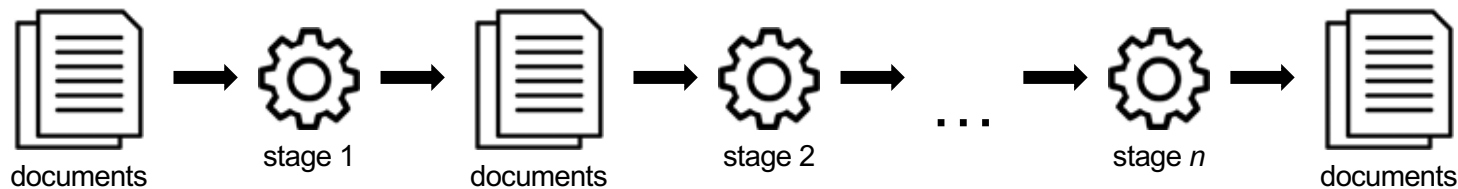
DATABASE MANAGEMENT SYSTEMS

LECTURE 15

MONGODB DBMS: AGGREGATION FRAMEWORK

AGGREGATION FRAMEWORK

In **MongoDB**, **analytics** on documents are performed via an **aggregation framework** which is based on the concept of a **data operation pipeline**.



- Input documents from a collection form a **data stream** that is passed through **one or more stages** (**processing units**) each of which **performs an operation** (**processing task**) on its inputs.

✎ **The inputs and outputs of all stages are documents.**

✎ **Each stage transforms the set of input documents into a new set of output documents based on the operation(s) performed in the stage.**

AGGREGATE METHOD

An aggregation is an array of stages.

`aggregate([pipeline]);`

The **aggregate** method **calculates aggregate values** for data in a collection.

➤ The *pipeline* parameter is a **sequence** of data aggregation **operations** or **stages**.

Commonly used data aggregation operations (stages)		SQL
\$count	counts the number of documents at this stage of the pipeline	count
\$group	groups documents by an identifier expression; applies optional accumulator expressions to each group; outputs one document for each group	group by
\$limit	passes the first <i>n</i> documents to the pipeline where <i>n</i> is the specified limit	fetch
\$lookup	performs a left outer join to a collection in the same database	join
\$match	filters the document stream against a predicate	where/having
\$project	restricts the document to contain only the specified fields	select
\$skip	skips the first <i>n</i> documents where <i>n</i> is the specified skip number	fetch ... offset
\$sort	reorders the document stream by a specified sort key	order by
\$unwind	outputs one document for each element in an array	N/A

✎ **A data aggregation operation can be used in multiple stages in a query.**

ACCUMULATORS

- Accumulators are operators that **maintain their state** as documents progress through an aggregation pipeline.
- Some common aggregation pipeline accumulators:
\$count **\$avg** **\$median** **\$sum** **\$max** **\$min** **\$stdDevPop**
\$addToSet adds an element to an array *only if* it is not already in the array
\$push adds an element to the end of an array; allows duplicate elements
- Most accumulators can be used in **\$project** or **\$group** stages.
 - In a **\$project** stage, accumulators perform calculations on **array field values** in a **single document**.
 - In a **\$group** stage, accumulators perform calculations on values across **multiple documents**.

AGGREGATION STAGE: \$PROJECT

\$project: {*projection*}

The **\$project** stage passes only the specified document fields to the next pipeline stage.

- *projection* is a **JSON** document that specifies the fields to include in or exclude from the input documents the same as that used in the **find** method. However, in an aggregation pipeline it cannot be an empty document.
- The specified fields can be existing fields from the input documents or newly computed fields.
- To add a new field or to rename an existing field, a field name is specified, and its value is set to some expression.

👉 The output documents contain only the projection fields.

Query: Find the districts of clients.

```
db.clients.aggregate([  
  {$project: {_id: 0, district: 1}}  
]);
```

Query {district: 'Central and Western'}
result: {district: 'Islands'}
 {district: null}
 {}

The **\$project** stage output contains only the district field (with duplicates retained).

AGGREGATION STAGE: \$MATCH

\$match: {*queryFilter*}

The **\$match** stage passes the documents that match the *queryFilter* to the next pipeline stage.

- *queryFilter* is a **JSON** document that specifies the selection criteria the same as that used in the **find** method.

👉 The output contains only those input documents that **match** *queryFilter*.

Query: Find only the client information for clients who live in Islands district.

```
db.clients.aggregate([
  {$match: {district: 'Islands'}},
  {$project: {_id: 0, accounts: 0, loans: 0, tags: 0}}
]);
```

SQL Note

The **\$match** stage *normally does not* allow aggregation expressions*.

* The **\$expr** operator, which uses aggregation expressions, is allowed to be used in a **\$match** stage.

Query result: {clientId: 4, name: 'Mary Kwan', hkid: 'M534109(3)', address: 'Hing Wah Street', district: 'Islands', rating: 6}

AGGREGATION STAGES: \$SORT; \$LIMIT; \$SKIP

\$sort: {*field*₁: *sortOrder*, *field*₂: *sortOrder*, ...}

The **\$sort** stage sorts all input documents according to the specified sort order(s) and outputs them.

- *sortOrder* can be **1** (ascending), or **-1** (descending) and is evaluated from left to right when sorting on multiple fields.

\$limit: *integer*

The **\$limit** stage restricts the number of output documents to *integer*.

\$skip: *integer*

The **\$skip** stage skips over *integer* number of input documents and outputs the remaining documents.

👉 **The content of the documents is not changed; they are either reordered (\$sort) or their number is reduced (\$limit and \$skip).**

MQL Note

Aggregation expressions **cannot be used** in the **\$sort**, **\$limit** and **\$skip** aggregation stages.

AGGREGATION STAGES: \$SORT; \$LIMIT; \$SKIP

(CONT'D)

Query: Find the client id, name and credit rating of any two clients with the highest credit ratings. Order the result by client name ascending.

```
db.clients.aggregate([
  { $sort: { rating: -1 } },      // order by rating descending
  { $limit: 2 },                 // retain only two highest ratings
  { $sort: { name: 1 } },        // order the result by client name
  { $project: { _id: 0, clientId: 1, name: 1, rating: 1 } }
]);
```

Query {clientId: 17, name: 'Pam Du', rating: 8}

result: {clientId: 3, name: 'Steven Chan', rating: 9}

AGGREGATION STAGE: \$GROUP

```
$group: {  
  _id: fieldPathList,  
  field: {accumulator: expression},  
  :  
}
```

A **\$group** stage **groups documents according to a group key** and **outputs one document** for each **unique group key**.

- **_id: *fieldPathList*** specifies the field(s) on which to group (i.e., the group key). If *fieldPathList* is specified as **null**, or any other constant value, the **\$group** stage returns a single document that aggregates values across all input documents. If *fieldPathList* consists of more than one field, then the list must be enclosed in curly brackets { } and each field path must be named, e.g.,
 _id: {clientId: '\$clientId', accountNo: '\$accounts.accountNo'}.
- **field: {accumulator: *expression*}** is optional and is computed using accumulator operators. A single **\$group** stage can contain multiple accumulator operators.

✎ **The output documents contain only the group key and the result of any accumulator operators.**

AGGREGATION STAGE: \$GROUP (CONTD)

Query: Find the minimum, maximum, average and median credit rating of all clients.

```
db.clients.aggregate([
  {$group: {
    _id: null,
    minRating: {$min: '$rating'},
    maxRating: {$max: '$rating'},
    avgRating: {$avg: '$rating'},
    medianRating: {$median: {input: '$rating', method: 'approximate'}}
  }},
  {$project: {_id: 0, min: '$minRating', max: '$maxRating',
    avg: {$trunc: ['$avgRating', 2]}, median: {$trunc: ['$medianRating', 2]}}}
]);
```

Query result: {min: 6, max: 9, avg: 7.25, median: 6}

SQL Notes

The **\$median** accumulator syntax is: **\$median**: {input: *number*, method: 'approximate'}

The **\$median** accumulator returns an approximation of the median, the 50th percentile, as a scalar value.

The **\$trunc** arithmetic expression operator syntax is **\$trunc**: [*number*, *place*] where *number* is the field value to truncate and *place* specifies the number of decimal places. If *place* is omitted, then *place* defaults to 0.

AGGREGATION STAGE: \$COUNT

\$count: {'*fieldName*'}

A **\$count** stage is used to count the number of documents that reached that stage.

- A single document is passed to the next stage that contains *fieldName* with the total count as its value.

Query: Find the number of clients with a rating greater than or equal to 8.

```
db.clients.aggregate([  
  {$match: {rating: {$gte: 8}}},  
  {$count: 'numLowRisk'}  
]);
```

Query result: {numLowRisk: 2}

SQL Note

When used as an accumulator within a **\$group** stage, **\$count** returns the number of documents in each group.

AGGREGATION STAGE: \$UNWIND

\$unwind: {path: *fieldPath*}

The **\$unwind** stage **deconstructs an array field** in the input documents and creates **one output document for each array element**.

- Each **output document is the same as the input document** except that **the value of the specified array field is replaced by one of the array elements** (i.e., one document is output *for each array element*).
- The **path** keyword is optional.
- If the field specified by *fieldPath* is not an array and is not **null** or **absent**, then one document is output with the field (i.e., the document is not changed).
- By default, no document is output if the array field value is **null**, **absent** or an empty array. This can be changed by specifying - **preserveNullAndEmptyArrays: true** - as an optional argument to the **\$unwind** stage.

 **The output contains one document for each element of the specified array.**

 **Since accumulators cannot be applied to array fields, unwinding is required to use accumulators on array fields.**

AGGREGATION STAGE: \$UNWIND (CONTD)

Query: Unwind client accounts.

Since client 17 has no account, it is not in the \$unwind stage output.

How to include it?

```
db.clients.aggregate([
  { $unwind: { path: '$accounts' } },
]);
```

≡

```
db.clients.aggregate([
  { $unwind: '$accounts' },
]);
```

Query result:

client 3

```
{clientId: 3, name: 'Steven Chan', hkid: 'C835618(4)', address: 'Mainmount Road', district: 'Central and Western', rating: 9,
  accounts: {accountNo: 'A-117', balance: 35000, branch: 'Star House'},
  loans: [{loanNo: 'L-171', amount: 90000, year: '2024', branch: 'Star House'},
    {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}],
  tags: ['senior', 'vip']}
```

client 4

```
{clientId: 3, name: 'Steven Chan', hkid: 'C835618(4)', address: 'Mainmount Road', district: 'Central and Western', rating: 9,
  accounts: {accountNo: 'A-120', balance: 80000, branch: 'Star House'},
  loans: [{loanNo: 'L-171', amount: 90000, year: '2024', branch: 'Star House'},
    {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}],
  tags: ['senior', 'vip']}

{clientId: 3, name: 'Steven Chan', hkid: 'C835618(4)', address: 'Mainmount Road', district: 'Central and Western', rating: 9,
  accounts: {accountNo: 'A-301', balance: 75000, branch: 'Pacific Place'},
  loans: [{loanNo: 'L-171', amount: 90000, year: '2024', branch: 'Star House'},
    {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}],
  tags: ['senior', 'vip']}

{clientId: 4, name: 'Mary Kwan', hkid: 'M534109(3)', address: 'Hing Wah Street', district: 'Islands', rating: 6,
  accounts: {accountNo: 'A-215', balance: 55000, branch: 'Diamond Hill'},
  ...}
```

AGGREGATION STAGE: \$UNWIND (CONTD)

Query: Unwind client accounts; include clients with no account.

```
db.clients.aggregate([
  {$unwind: {path: "$accounts", preserveNullAndEmptyArrays: true}}
]);
```

Query result:

client 3	{clientId: 3, name: 'Steven Chan', hkid: 'C835618(4)', address: 'Mainmount Road', district: 'Central and Western', rating: 9, accounts: {accountNo: 'A-117', balance: 35000, branch: 'Star House'}, loans: [{loanNo: 'L-171', amount: 90000, year: '2024', branch: 'Star House'}, {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}], tags: ['senior', 'vip']}
	⋮
	⋮
client 4	{clientId: 4, name: 'Mary Kwan', hkid: 'M534109(3)', address: 'Hing Wah Street', district: 'Islands', rating: 6, accounts: {accountNo: 'A-215', balance: 55000, branch: 'Diamond Hill'},
	⋮
client 8	{clientId: 8, name: 'Isaac Li', hkid: 'K917263(6)', address: 'Man Lai Court', district: null, rating: 6, accounts: [{accountNo: 'A-120', balance: 80000, branch: 'Star House'}]},
client 17	{clientId: 17, name: 'Pam Du', hkid: 'W251984(6)', address: 'Tsing Yi Road', rating: 8, loans: [{loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}], tags: ['vip']}



AGGREGATION STAGE: \$UNWIND (CONTD)

Query: Find the average account balance at the Pacific Place branch.

```
db.clients.aggregate([
  {$unwind: {path: '$accounts'}},
  {$match: {'accounts.branch': 'Pacific Place'}},
  {$group: { // calculate the average account balance
    _id: null,
    avgBalance: {$avg: '$accounts.balance'}
  }},
  {$project: {_id: 0, branch: '$_id', avgBalance: '$avgBalance'}}
]);
```

Query result:
{avgBalance: 60000}

Why a
different result?

Does it matter if we switch the \$unwind and \$match stages?

```
db.clients.aggregate([
  {$match: {'accounts.branch': 'Pacific Place'}},
  {$unwind: {path: '$accounts'}},
  {$group: { // calculate the average account balance
    _id: null,
    avgBalance: {$avg: '$accounts.balance'}
  }},
  {$project: {_id: 0, branch: '$_id', avgBalance: '$avgBalance'}}
]);
```

Query result:
{avgBalance: 58000}

AGGREGATION STAGE: \$UNWIND (CONTD)

Query: Find the average account balance at the Pacific Place branch.

```
db.clients.aggregate([
  {$unwind: {path: '$accounts'}},
  {$match: {'accounts.branch': 'Pacific Place'}},
  {$project: {_id: 0, accounts: 1}}
]);
```

\$unwind followed by **\$match**

The **\$match** stage includes only accounts at the Pacific Place branch.

Query {accounts: {accountNo: 'A-301', balance: 75000, branch: 'Pacific Place'}}

result: {accounts: {accountNo: 'A-322', balance: 45000, branch: 'Pacific Place'}}

```
db.clients.aggregate([
  {$match: {'accounts.branch': 'Pacific Place'}},
  {$unwind: {path: '$accounts'}},
  {$project: {_id: 0, accounts: 1}}
]);
```

\$match followed by **\$unwind**

The **\$match** stage can include accounts at branches other than the Pacific Place branch (see the previous lecture **Querying Arrays**).

Query {accounts: {accountNo: 'A-117', balance: 35000, branch: 'Star House'}}

result: {accounts: {accountNo: 'A-120', balance: 80000, branch: 'Star House'}}

{accounts: {accountNo: 'A-301', balance: 75000, branch: 'Pacific Place'}}

{accounts: {accountNo: 'A-215', balance: 55000, branch: 'Diamond Hill'}}

{accounts: {accountNo: 'A-322', balance: 45000, branch: 'Pacific Place'}}

AGGREGATION STAGES: \$UNWIND AND \$GROUP

Query: Find, for each client, their id and the average and total balance of all the accounts they hold. Order the result by client id ascending.

```
db.clients.aggregate([
  { $unwind: '$accounts', // required to use accumulators on the balance field
  { $group: {
    _id: '$clientId', // group by clientId
    average: { $avg: '$accounts.balance' },
    total: { $total: '$accounts.balance' }
  } },
  { $project: { _id: 0, clientId: '$_id', average: { $trunc: ['$average', 2],
    total: '$total' } },
  { $sort: { clientId: 1 } }
]);
```

Query {clientId: 3, average: 6333.33, total: 190000}

result: {clientId: 4, average: 50000, total: 100000}

{clientId: 8, average: 80000, total: 80000}

How to show the name rather than the client id?

AGGREGATION STAGES: \$UNWIND AND \$GROUP

(CONT'D)

Query: Find, for each client, their name and the average, median and total balance of all the accounts they hold. Order the result by client id ascending.

```
db.clients.aggregate([
  { $unwind: '$accounts' },
  { $group: {
    _id: { clientId: '$clientId', name: '$name' }, // group by clientId and name
    average: { $avg: '$accounts.balance' },
    total: { $total: '$accounts.balance' }
  } },
  { $project: { _id: 0, name: '$_id.name', average: { $trunc: ['$average', 2],
    total: '$total' } },
  { $sort: { '_id.clientId': 1 } }
]);
```

Query {name: 'Steven Chan', average: 6333.33, total: 190000}

result: {name: 'Mary Kwan', average: 50000, total: 100000}

{name: 'Isaac Li', average: 80000, total: 80000}

AGGREGATION STAGES: \$UNWIND AND \$GROUP (CONT'D)

Query: Find the total account balance of each branch. Order the result by branch ascending.

Is this a correct solution?
No! Why?

```
db.clients.aggregate([
  { $unwind: '$accounts' },
  { $group: {
    _id: '$accounts.branch', // group by branch
    total: { $sum: '$accounts.balance' }
  } },
  { $project: { _id: 0, branch: '$_id', total: '$total' } },
  { $sort: { branch: 1 } }
]);
```

Query result: {branch: 'Diamond Hill', total: 55000}
{branch: 'Pacific Place', total: 120000}
{branch: 'Star House', total: 195000}

Expected result: {branch: 'Diamond Hill', total: 55000}
{branch: 'Pacific Place', total: 120000}
{branch: 'Star House', total: 115000}

What's the problem?

ACCUMULATOR OPERATOR: \$ADDTOSET

\$addToSet: {*field*₁: *value*₁, ...}, {*modifiers*}

The **\$addToSet** operator adds a value to an array *only if* the value is not already in the array.

- If *field* is absent from the document, then an array field named *field* is created with the specified *value* as its element.
- The **\$addToSet** operator only ensures that there are no duplicate elements added to the set; it does not affect existing duplicate elements.
- The optional **\$each** modifier adds multiple values to the array field. The multiple values are specified as an array value for the **\$each** modifier, i.e.,
\$addToSet: {*field*: {**\$each**: [*value*₁, *value*₂, ...]}}

☞ **The output contains the array field added by the \$addToSet operator in addition to any other fields added by other stage operators.**

☞ **The \$addToSet operator can be used to remove duplicate values (i.e., keep only unique values).**

ACCUMULATOR OPERATOR: \$ADDTOSET (CONTD)

Query: Find the total account balance of each branch. Order the result by branch ascending.

```
db.clients.aggregate([
  {$unwind: {path: '$accounts'}},
  {$group: { // retain only the unique accounts (remove duplicates)
    _id: null,
    uniqueAccounts: {$addToSet: {accountNo: '$accounts.accountNo',
                                balance: '$accounts.balance',
                                branch: '$accounts.branch'}}
  }},
  ...
])
```

Query result after \$group stage:

```
{_id: null, uniqueAccounts: [{accountNo: 'A-215', balance: 55000, branch: 'Diamond Hill'},
                             {accountNo: 'A-117', balance: 35000, branch: 'Star House'},
                             {accountNo: 'A-301', balance: 75000, branch: 'Pacific Place'},
                             {accountNo: 'A-120', balance: 80000, branch: 'Star House'},
                             {accountNo: 'A-322', balance: 45000, branch: 'Pacific Place'}]}
```

Account A-120
appears only
once in the result.

👉 **Now need to unwind** uniqueAccounts **array to sum balances.**

ACCUMULATOR OPERATOR: \$ADDTOSET (CONTD)

Query: Find the total account balance of each branch. Order the result by branch ascending.

```
{ $unwind: { path: '$uniqueAccounts' },
```

Query result after \$unwind stage:

```
{_id: null, uniqueAccounts: {accountNo: 'A-215', balance: 55000, branch: 'Diamond Hill'}}
{_id: null, uniqueAccounts: {accountNo: 'A-117', balance: 35000, branch: 'Star House'}}
{_id: null, uniqueAccounts: {accountNo: 'A-301', balance: 75000, branch: 'Pacific Place'}}
{_id: null, uniqueAccounts: {accountNo: 'A-120', balance: 80000, branch: 'Star House'}}
{_id: null, uniqueAccounts: {accountNo: 'A-322', balance: 45000, branch: 'Pacific Place'}}
```

👉 **Now need to group again by branch.**

ACCUMULATOR OPERATOR: \$ADDTOSET (CONTD)

Query: Find the total account balance of each branch. Order the result by branch ascending.

```
⋮  
{ $group: {  
  _id: '$uniqueAccounts.branch', // group by branch  
  total: { $sum: '$uniqueAccounts.balance' }  
}},  
{ $project: { _id: 0, branch: '$_id', total: '$total' } },  
{ $sort: { branch: 1 } }  
]);
```

Query {branch: 'Diamond Hill', total: 55000}
result: {branch: 'Pacific Place', total: 120000}
 {branch: 'Star House', total: 115000}

ACCUMULATOR OPERATOR: \$PUSH

\$push: {*field*₁: *value*₁, ...}, {*modifiers*}

The **\$push** operator appends a specified value to an array.

- If *field* is absent from the document, then an array field named *field* is created with the specified *value(s)* as its element.
- Elements are added at the end of the array and *duplicate elements are retained*.
- If a *value* is an array, then the whole array is added as a single array element.
- The following optional modifiers are allowed:
 - **\$each** adds multiple values to the array *field*. The multiple values are specified as an array value for the **\$each** modifier, i.e.,
\$push: {*field*: {**\$each**: [*value*₁, *value*₂, ...]}}.
 - **\$slice** limits the number of array elements; requires use of the **\$each** modifier.
 - **\$sort** orders the elements of the array; requires use of the **\$each** modifier.
 - **\$position** specifies the location in the array at which to insert the new elements; requires use of the **\$each** modifier.

👉 **The output contains the array field added by the \$push operator in addition to any other fields added by other stage operators.**

ACCUMULATOR OPERATOR: **\$PUSH** (CONT'D)

Query: Find, for each branch, the branch name, client id, client name, client account number and client account balance of the clients with the largest single account balance at the branch.

```
db.clients.aggregate([
  {$unwind: {path: '$accounts'}},
  {$group: {
    _id: '$accounts.branch',
    maxBalance: {$max: '$accounts.balance'},
    clients: {$push: {clientId: '$clientId',
                      name: '$name',
                      accountNo: '$accounts.accountNo',
                      balance: '$accounts.balance'}}
  }},
  ...
])
```

Query result after \$group stage:

```
{_id: 'Diamond Hill', maxBalance: 55000, clients: [{clientId: 4, name: 'Mary Kwan', accountNo: 'A-215', balance: 55000}]}
{_id: 'Pacific Place', maxBalance: 75000, clients: [{clientId: 3, name: 'Steven Chan', accountNo: 'A-301', balance: 75000},
  {clientId: 4, name: 'Mary Kwan', accountNo: 'A-322', balance: 45000}]}
{_id: 'Star House', maxBalance: 80000, clients: [{clientId: 3, name: 'Steven Chan', accountNo: 'A-117', balance: 35000},
  {clientId: 3, name: 'Steven Chan', accountNo: 'A-120', balance: 80000},
  {clientId: 8, name: 'Isaac Li', accountNo: 'A-120', balance: 80000}]}
```

ACCUMULATOR OPERATOR: **\$PUSH** (CONT'D)

Query: Find, for each branch, the branch name, client id, client name, client account number and client account balance of the clients with the largest single account balance at the branch.

```
{ $unwind: { path: '$clients' },
```

```
⋮  
⋮  
⋮
```

Query result after \$unwind stage:

```
{_id: 'Diamond Hill', maxBalance: 55000, clients: {clientId: 4, name: 'Mary Kwan', accountNo: 'A-215', balance: 55000}}  
{_id: 'Pacific Place', maxBalance: 75000, clients: [{clientId: 3, name: 'Steven Chan', accountNo: 'A-301', balance: 75000},  
{_id: 'Pacific Place', maxBalance: 75000, clients: {clientId: 4, name: 'Mary Kwan', accountNo: 'A-322', balance: 45000}}  
{_id: 'Star House', maxBalance: 80000, clients: {clientId: 3, name: 'Steven Chan', accountNo: 'A-117', balance: 35000}},  
{_id: 'Star House', maxBalance: 80000, clients: {clientId: 3, name: 'Steven Chan', accountNo: 'A-120', balance: 80000}},  
{_id: 'Star House', maxBalance: 80000, clients: {clientId: 8, name: 'Isaac Li', accountNo: 'A-120', balance: 80000}}
```

ACCUMULATOR OPERATOR: \$PUSH (CONTD)

Query: Find, for each branch, the branch name, client id, client name, client account number and client account balance of the clients with the largest single account balance at the branch.

```

      :
      :
      :
// retain only the clients with the maximum branch balance
{$match: {$expr: {$eq: ['$maxBalance', '$clients.balance']}}},
{$project: {_id: 0, branch: '$_id', clientId: '$clients.clientId', name: '$clients.name',
accountNo: '$clients.accountNo', balance: '$clients.balance'}}
]);
```

Query {branch: 'Diamond Hill', clientId: 4, name: 'Mary Kwan', accountNo: 'A-215', balance: 55000}
result: {branch: 'Pacific Place', clientId: 3, name: 'Steven Chan', accountNo: 'A-301', balance: 75000}
 {branch: 'Star House', clientId: 3, name: 'Steven Chan', accountNo: 'A-120', balance: 80000}
 {branch: 'Star House', clientId: 8, name: 'Isaac Li', accountNo: 'A-120', balance: 80000}

👉 **The \$push operator can be used to retain input document fields during grouping.**

ACCUMULATOR OPERATOR: **\$PUSH** (CONTD)

Query: Find the client id, name and credit rating of all clients with the third highest credit rating. Order the result by client name ascending.

```
db.clients.aggregate([
  {
    $group: {
      _id: '$rating',
      clients: {
        $push: {
          clientId: '$clientId',
          name: '$name'
        }
      }
    },
    $sort: { _id: -1 },
    $skip: 2,
    $limit: 1,
    // order the clients by client name
    $project: {
      _id: 0,
      rating: '$_id',
      clients: {
        $sortBy: {
          input: '$clients',
          sortBy: { name: 1 }
        }
      }
    }
  }
]);
```

Query {rating: 6, clients: [{clientId: 8, name: 'Isaac Li'},
result: {clientId: 4, name: 'Mary Kwan'}]}

AGGREGATION STAGE: \$LOOKUP

```
$lookup: {  
  from: collectionToJoin,  
  localField: fieldFromInputDocument,  
  foreignField: fieldFromCollectionToJoin,  
  as: outputArrayField  
}
```

SQL Notes

1. Aggregation expressions **cannot be used** in the **\$lookup** aggregation stage.
2. All field and collection names must be enclosed in quotes.

The **\$lookup** stage performs a **left outer join** on the input documents to a collection in the **same database** by **adding** a **new array field** to each input document that contains the matching documents from *collectionToJoin*.

- *collectionToJoin* specifies a collection in the same database to perform the join with.
- *fieldFromInputDocument* specifies a field from the input document.
- *fieldFromCollectionToJoin* specifies a field in *collectionToJoin*.
- *outputArrayField* specifies the name of the added array field which will contain the matching documents from *collectionToJoin*.

👉 **The output documents are the same as the input documents except that an array field is added containing the matching documents from the collection being joined.**

AGGREGATION STAGE: \$LOOKUP (CONT'D)

Query: Find the name and district of clients who have an account in the same district in which they live.

```
db.clients.aggregate([
  {$unwind: {path: '$accounts'}},
  {$lookup: {
    from: 'branches',
    localField: 'accounts.branch', // match the accounts branch name
    foreignField: 'branch',        // with the branches branch name
    as: 'branchInfo'
  }},
  ...
])
```

Query result after \$unwind and \$lookup stages:

```
{clientId: 3, name: 'Steven Chan', hkid: 'C835618(4)', address: 'Mainmount Road', district: 'Central and Western', rating: 9,
  accounts: {accountNo: 'A-117', balance: 35000, branch: 'Star House'},
  loans: [{loanNo: 'L-171', amount: 90000, year: '2024', branch: 'Star House'},
    {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}],
  tags: ['senior', 'vip'],
  branchInfo: [{branch: 'Star House', district: 'Yau Tsim Mong', liabilities: 115000, assets: 115000, accounts: ['A-117', 'A-120'], loans: ['L-171']}]}
{clientId: 3, name: 'Steven Chan', hkid: 'C835618(4)', address: 'Mainmount Road', district: 'Central and Western', rating: 9,
  accounts: {accountNo: 'A-120', balance: 80000, branch: 'Star House'},
  loans: ....
  tags: ...
  branchInfo: [{branch: 'Star House', district: 'Yau Tsim Mong', liabilities: 115000, assets: 115000, accounts: ['A-117', 'A-120'], loans: ['L-171']}]}
}
{clientId: 3, name: 'Steven Chan', hkid: 'C835618(4)', address: 'Mainmount Road', district: 'Central and Western', rating: 9,
  accounts: {accountNo: 'A-301', balance: 75000, branch: 'Pacific Place'},
  loans: ....
  tags: ...
  branchInfo: [{branch: 'Pacific Place', district: 'Central and Western', liabilities: 120000, assets: 40000, accounts: ['A-301', 'A-322'], loans: ['L-321']}]}
}
{clientId: 4, name: 'Mary Kwan', hkid: 'M534109(3)', address: 'Hing Wah Street', district: 'Islands', rating: 6,
  accounts: {accountNo: 'A-215', balance: 55000, branch: 'Diamond Hill'},
  loans: [{loanNo: 'L-321', amount: 40000, year: '2024', branch: 'Pacific Place'},
    {loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}],
  tags: ['senior'],
  branchInfo: [{branch: 'Diamond Hill', district: 'Kwun Tong', liabilities: 55000, assets: 0, accounts: ['A-215']}]}
}
{clientId: 4, name: 'Mary Kwan', hkid: 'M534109(3)', address: 'Hing Wah Street', district: 'Islands', rating: 6,
  accounts: {accountNo: 'A-322', balance: 45000, branch: 'Pacific Place'},
  :
  branchInfo: [{branch: 'Pacific Place', district: 'Central and Western', liabilities: 120000, assets: 40000, accounts: ['A-301', 'A-322'], loans: ['L-321']}]}
}
{clientId: 8, name: 'Isaac Li', hkid: 'K917263(6)', address: 'Man Lai Court', district: null, rating: 6,
  accounts: {accountNo: 'A-120', balance: 80000, branch: 'Star House'},
  branchInfo: [{branch: 'Star House', district: 'Yau Tsim Mong', liabilities: 115000, assets: 115000, accounts: ['A-117', 'A-120'], loans: ['L-171']}]}
}
```



AGGREGATION STAGE: \$LOOKUP (CONT'D)

Query: Find the name and district of clients who have an account in the same district in which they live.

```
⋮  
{ $unwind: { path: '$branchInfo' }, // required to use $expr operator  
  { $match: { $expr: { $eq: ['$district', '$branchInfo.district'] } } },  
  { $project: { _id: 0, name: 1, district: 1 } }  
});
```

Query result: {name: 'Steven Chan', district: 'Central and Western'}

MONGODB DBMS: SUMMARY

- **MongoDB** is an open-source, schemaless NoSQL DBMS designed for horizontal scaling.
- Data is stored in JSON-like collections of documents.
- Unlike relational databases that rely on foreign keys and join operations, **MongoDB** supports:
 - references which use unique identifiers to establish relationships between documents.
 - embedded data which store related data in a single document (reducing the need for joins).
- The **MongoDB** query language (**MQL**) supports basic database operations as well as analytics on documents via an aggregation framework.